



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2113 DISEÑO DETALLADO DE SOFTWARE

Proyecto: Octopath Traveler

Javiera Ignacia Pinto Santa María
Matías Andrés Poblete Farías

Introducción

La librería `Octopath_Traveler_GUI` incluye una clase, 5 interfaces y 2 enumeraciones:

- `OTGUI` es una clase que te permitirá abrir una ventana y cambiar lo que aparece en ella.
- `ITraveler` es una interfaz con métodos para obtener la información del viajero (e.j. su nombre, HP, etc).
- `IBeast` es una interfaz con métodos para obtener la información de la bestia (e.j. su nombre, HP, etc).
- `IState` es una interfaz con métodos para obtener la información del juego, como las opciones que tiene disponible para elegir el jugador, la cantidad de turnos, etc.
- `Option` es una enumeración que tiene los distintos tipos de opciones sobre qué información tiene que seleccionar el jugador.
- `IClickedElement` es una interfaz con métodos para obtener la información del último elemento que se le hizo click en la ventana, como el texto o el tipo de unidad seleccionada.
- `ClickElementType` es una enumeración que tiene los distintos tipos de elementos que puede ser `IClickedElement`.
- `IWinner` es una interfaz con métodos para mostrar el equipo de ganó y bajo qué condiciones.

Para utilizar la librería, puedes comenzar desde la nueva versión del código base o seguir las instrucciones descritas en el Apéndice [A](#)

1. Interfaces `IState`, `ITraveler` y `IBeast`

Cualquier clase que implemente la interfaz `IState` tendrá que implementar estos métodos:

```
1 public interface IState
2 {
3     public IEnumerable<ITraveler> Travelers { get; }
4     public IEnumerable<IBeast> Beasts { get; }
5     public IEnumerable<string> Options { get; }
6     public Option Option { get; }
7     public IEnumerable<string> CurrentRoundTurns { get; }
8     public IEnumerable<string> NextRoundTurns { get; }
9 }
```

El método `void Update(IState state)` de la clase `OTGUI` recibe un objeto que implemente `IState`, por lo tanto, para utilizar este método, tendrás que tener una clase en tu código que la implemente.

Dado que `IState` tiene los valores `Travelers` y `Beasts` tendrás que tener una clase que implemente dichas interfaces:

```
1 public interface ITraveler
2 {
3     public string Name { get; }
4     public int HP { get; }
5     public int MaxHP { get; }
6     public int SP { get; }
7     public int MaxSP { get; }
8     public int BoostPoints { get; }
9 }
```

```
1 public interface IBeast
2 {
3     public string Name { get; }
4     public int HP { get; }
5     public int MaxHP { get; }
6     public int Shields { get; }
7 }
```

Además, `Option` es una enumeración que contiene los siguientes miembros:

```
1 public enum Option
2 {
3     Action,
4     Weapon,
5     Skill,
6     BoostPoints,
7     Target
8 }
```

2. Interfaz `IClickedElement`

La interfaz `IClickedElement` expone los siguientes parámetros:

```
1 public interface IClickedElement
2 {
3     string Text { get; }
4     ClickElementType Type { get; }
5 }
```

No es necesario que crees clases que implementen esta interfaz, ya que esta interfaz sólo es utilizada como retorno del método `IClickedElement GetClickedElement()` de la clase `OTGUI`.

Cabe destacar que `Type` contiene una instancia de tipo `ClickElementType`, la cual es una enumeración que contiene los siguientes miembros:

```
1 public enum ClickElementType
2 {
3     Traveler,
4     Beast,
5     Button
6 }
```

3. Interfaz `IWinner`

La interfaz `IWinner` expone los siguientes métodos:

```

1 public interface IWinner
2 {
3     public WinnerOption WinnerOption { get; }
4     public IEnumerable<string> Team { get; }
5 }

```

El método `ShowWinner(IWinner winner)` de la clase `OTGUI` recibe un objeto que implemente `IWinner`, por lo tanto, para utilizar este método tendrás que tener una clase en tu código que la implemente.

Por último, `WinnerOption` es una enumeración que contiene los siguiente miembros:

```

1 public enum WinnerOption
2 {
3     Travelers,
4     Beasts,
5     RunAway
6 }

```

4. Clase OTGUI

Como se dijo anteriormente, esta clase permite abrir una ventana y actualizar su contenido. Para abrir una ventana, primero se crea el objeto de tipo `OTGUI` y luego se llama al método `Start(...)`.

- `void Start(Action startProgramCallback)`: este método inicia una ventana y luego invoca `startProgramCallback`. Notar que el parámetro `startProgramCallback` es de tipo `Action`. Este es un tipo de dato utilizado para encapsular funciones que no retornan nada y que no reciben argumentos.

Ocurre que MacOS no permite crear ventanas desde threads que no sean el principal del programa. Esto nos obliga a que, luego de creada la ventana, esa ventana se quede con el thread principal de la aplicación (y no lo suelta hasta que la ventana se cierra). Para que tu programa siga funcionando, nosotros invocamos `startProgramCallback` desde el thread secundario. Por lo mismo, considera que cuando llames a `Start(Action startProgramCallback)` tu código continuará desde el método encapsulado en `startProgramCallback`.

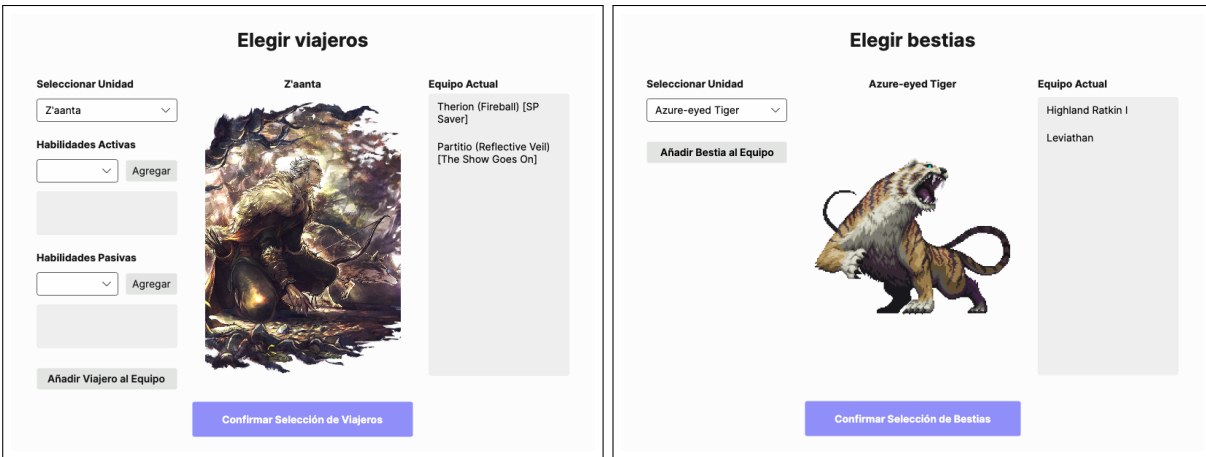
Por ejemplo, este `Program.cs` abre la ventana y luego llama a `Main()`, que no hace nada.

```

1 using Octopath_Traveler_GUI;
2
3 OTGUI gui = new OTGUI();
4 gui.Start(Main);
5
6 void Main() {
7     // Por ahora, no hace nada.
8 }

```

Si corres este programa se abre la ventana que permite elegir el equipo de los viajeros y, al presionar el botón “Confirmar Selección de Viajeros”, abrirá la ventana de selección del equipo de bestias. Pero dado que el método `Main` está vacío, no ocurre nada al apretar el botón “Confirmar Selección de Bestias”



Para obtener los equipos ingresados, puedes utilizar los métodos:

- `string[] GetTravelersTeam()`: retorna una lista de `strings` con la información elegida por el usuario para los viajeros seleccionados.
- `string[] GetBeastsTeam()`: retorna una lista de `strings` con la información elegida por el usuario para las bestias seleccionadas.

Por ejemplo, este código abre una ventana y permite seleccionar a los equipos, luego, muestra la información en consola.

```

1 var gui = new OTGUI();
2 gui.Start(Main);
3
4 void Main()
5 {
6     string[] travelersTeam = gui.GetTravelersTeam();
7     ShowTeam(travelersTeam);
8     string[] beastsTeam = gui.GetBeastsTeam();
9     ShowTeam(beastsTeam);
10 }
11
12 void ShowTeamInfo(string[] team)
13 {
14     foreach (var unit in team)
15     {
16         Console.WriteLine(unit);
17     }
18 }

```

Al ejecutar este código, el programa se quedará en la línea 6 esperando hasta que el primer jugador presione “Confirmar Selección de Viajeros”. En ese momento, `GetTravelersTeam()` retornará una lista de `string` que contiene la información seleccionada por el jugador.

En este caso, presionar “Confirmar Selección de Viajeros” hará que el método `GetTravelersTeam()` imprima esto en consola:

```
1 Tressa (Luminescence) [Elemental Augmentation]
2 Z'aanta (Luminescence)
3 Partitio [Summon Strength]
4 Therion
```

Una vez seleccionados los equipos, hay dos métodos a los que se puede llamar:

- `void ShowInvalidTeams()`: muestra una vista con el mensaje “Al menos uno de los equipos es inválido”.
- `void Update(IState state)`: actualiza el contenido de la ventana en base a `state`, en particular, muestra la vista principal del juego (tablero, turnos, opciones, etc.).

Como se mencionó anteriormente, para poder utilizar el método `Update(...)` es necesario crear clases que implementen las interfaces que utiliza este método. Supongamos que creamos las siguientes clases¹:

```
1 public class Traveler(string name, int hp, int maxHp, int sp, int maxSp, int boostPoints)
2     : ITraveler
3 {
4     public string Name { get; set; } = name;
5     public int HP { get; set; } = hp;
6     public int MaxHP { get; set; } = maxHp;
7     public int SP { get; set; } = sp;
8     public int MaxSP { get; set; } = maxSp;
9     public int BoostPoints { get; set; } = boostPoints;
10 }
```

```
1 public class Beast(string name, int hp, int maxHp, int shields)
2     : IBeast
3 {
4     public string Name { get; set; } = name;
5     public int HP { get; set; } = hp;
6     public int MaxHP { get; set; } = maxHp;
7     public int Shields { get; set; } = shields;
8 }
```

```
1 public class State(IEnumerable<ITraveler> travelers, IEnumerable<IBeast> beasts, IEnumerable
2     <string> options, Option option)
3     : IState
4 {
5     public IEnumerable<ITraveler> Travelers { get; set; } = travelers;
6     public IEnumerable<IBeast> Beasts { get; set; } = beasts;
7
8     public IEnumerable<string> Options { get; set; } = options;
9
10    public Option Option { get; set; } = option;
11
12    public IEnumerable<string> CurrentRoundTurns { get; set; } =
```

¹Les recomendamos no utilizar las clases que ya tengan creadas para las unidades, creen nuevas clases que implementen las interfaces para la GUI

```

12     [
13         "Tressa",
14         "Partitio",
15         "Therion",
16         "Z'aanta",
17         "Mattias?",
18         "Meep"
19     ];
20     public IEnumerable<string> NextRoundTurns { get; set; } =
21     [
22         "Mattias?",
23         "Therion",
24         "Tressa",
25         "Z'aanta",
26         "Partitio",
27         "Meep"
28     ];
29 }

```

Ahora, en el Program.cs creamos una instancia de State cualquiera, ignorando los equipos seleccionados por el usuario:

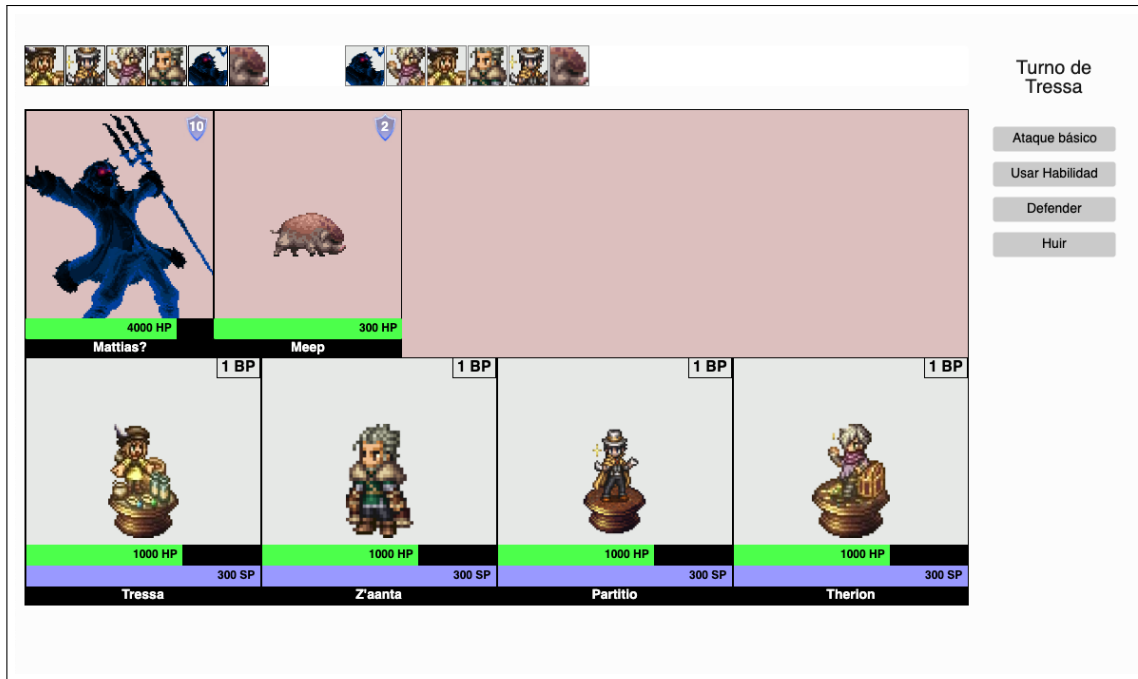
```

1 var gui = new OTGUI();
2 gui.Start(Main);
3
4 void Main()
5 {
6     string[] travelersTeam = gui.GetTravelersTeam();
7     string[] beastsTeam = gui.GetBeastsTeam();
8     var tressa = new Traveler("Tressa", 1000, 1500, 300, 300, 1);
9     var zaanta = new Traveler("Z'aanta", 1000, 1500, 300, 300, 1);
10    var partitio = new Traveler("Partitio", 1000, 1500, 300, 300, 1);
11    var therion = new Traveler("Therion", 1000, 1500, 300, 300, 1);
12    IEnumerable<Traveler> travelers = [tressa, zaanta, partitio, therion];
13    var mattias = new Beast("Mattias?", 4000, 5000, 10);
14    var meep = new Beast("Meep", 300, 300, 2);
15    IEnumerable<Beast> beasts = [mattias, meep];
16    IEnumerable<string> options = ["Ataque básico", "Usar Habilidad", "Defender", "Huir"];
17    var state = new State(travelers, beasts, options, Option.Action);
18    gui.Update(state);
19 }

```

Este código mostrará las ventanas para elegir los equipos y luego mostrará esta ventana²:

²En este ejemplo, los turnos están hardcodeados. En la práctica, no debería ser así.



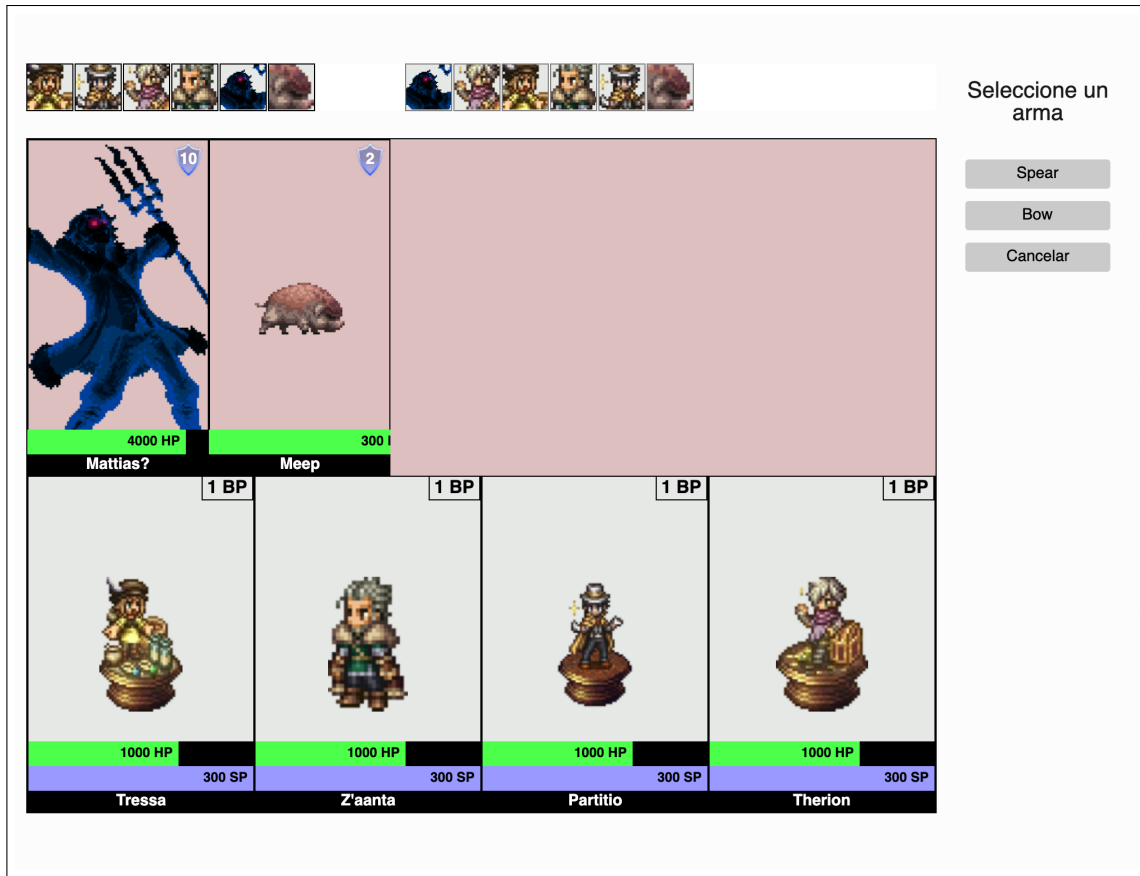
La enumeración `Option` tiene cinco miembros que se pueden utilizar para actualizar esta vista y de esta dependen para el texto que se muestra. Si cambiásemos el código anterior para dar la opción de usar `Weapon` en vez de las acciones:

```

1 var gui = new OTGUI();
2 gui.Start(Main);
3
4 void Main()
5 {
6     string[] travelersTeam = gui.GetTravelersTeam();
7     string[] beastsTeam = gui.GetBeastsTeam();
8     var tressa = new Traveler("Tressa", 1000, 1500, 300, 300, 1);
9     var zaanta = new Traveler("Z'aanta", 1000, 1500, 300, 300, 1);
10    var partitio = new Traveler("Partitio", 1000, 1500, 300, 300, 1);
11    var therion = new Traveler("Therion", 1000, 1500, 300, 300, 1);
12    IEnumerable<Traveler> travelers = [tressa, zaanta, partitio, therion];
13    var mattias = new Beast("Mattias?", 4000, 5000, 10);
14    var meep = new Beast("Meep", 300, 300, 2);
15    IEnumerable<Beast> beasts = [mattias, meep];
16    IEnumerable<string> options = ["Spear", "Bow", "Cancelar"];
17    var state = new State(travelers, beasts, options, Option.Weapon);
18    gui.Update(state);
19 }

```

La vista se actualizaría a lo siguiente:



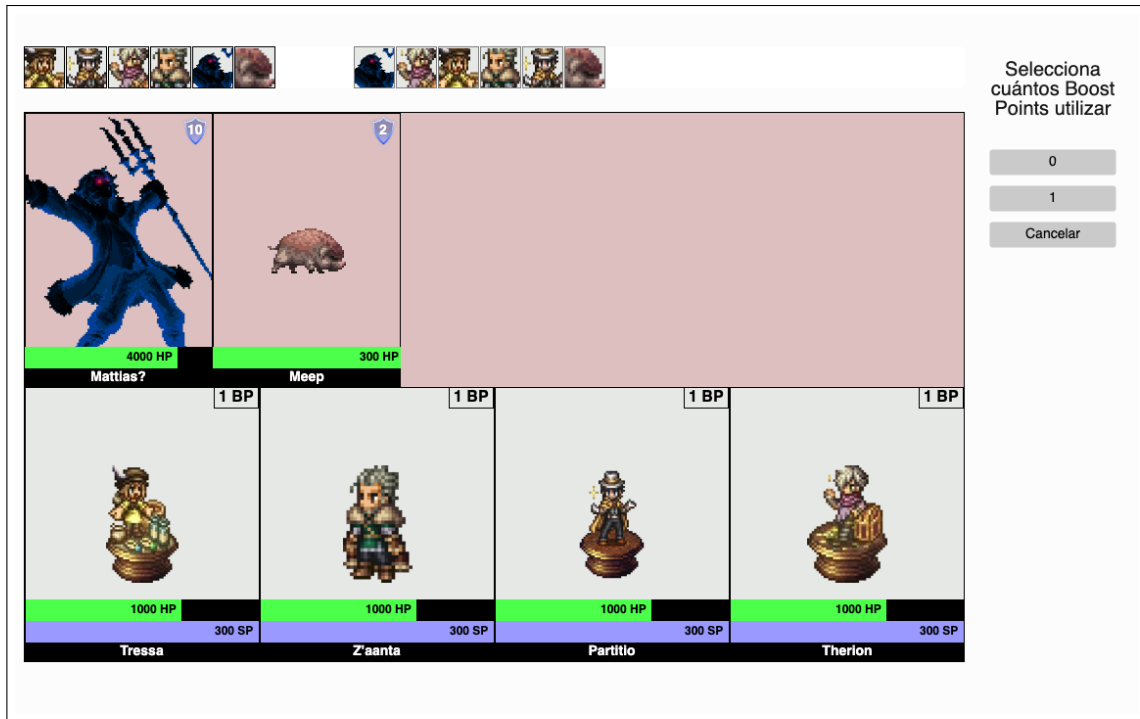
En cambio, si cambiásemos el código para mostrar las opciones de BoostPoints:

```

1 var gui = new OTGUI();
2 gui.Start(Main);
3
4 void Main()
5 {
6     string[] travelersTeam = gui.GetTravelersTeam();
7     ShowTeam(travelersTeam);
8     string[] beastsTeam = gui.GetBeastsTeam();
9     ShowTeam(beastsTeam);
10    var tressa = new Traveler("Tressa", 1000, 1500, 300, 300, 1);
11    var zaanta = new Traveler("Z'aanta", 1000, 1500, 300, 300, 1);
12    var partitio = new Traveler("Partitio", 1000, 1500, 300, 300, 1);
13    var therion = new Traveler("Therion", 1000, 1500, 300, 300, 1);
14    IEnumerable<Traveler> travelers = [tressa, zaanta, partitio, therion];
15    var mattias = new Beast("Mattias?", 4000, 5000, 10);
16    var meep = new Beast("Meep", 300, 300, 2);
17    IEnumerable<Beast> beasts = [mattias, meep];
18    IEnumerable<string> options = ["0", "1", "Cancelar"];
19    var state = new State(travelers, beasts, options, Option.BoostPoints);
20    gui.Update(state);
21 }

```

La vista se actualizaría a lo siguiente:



Desde esta ventana podemos interactuar con todos los botones (Button), viajeros (Traveler) y bestias en el tablero (Beast).

Para poder obtener información del usuario puedes utilizar el método `GetClickedElement()`, el cual retorna una instancia de `IClickedElement`, el cual retorna el tipo de elemento seleccionado (Type) junto al contenido del elemento seleccionado (Text).

Cabe destacar que `GetClickedElement()` **espera** a que el usuario haga *click* en algún elemento y retorna. Por lo tanto, su retorno nunca será null. Por otro lado, dado que en ocasiones ustedes querrán que el usuario seleccione ciertos elementos, probablemente necesitarán algo de este estilo:

```
1 IClickedElement clickedElement;
2 do {
3     clickedElement = gui.GetClickedElement();
4 } while (!(clickedElement.Type == ClickedElementType.Button && clickedElement.Text == "
    Ataque básico"));
```

Este bucle sólo se romperá cuando el elemento retornado por `GetClickedElement` cumpla las condiciones correspondientes.

Por ejemplo, si modificamos el método `Main()` mostrado anteriormente, y le agregamos las siguientes líneas:

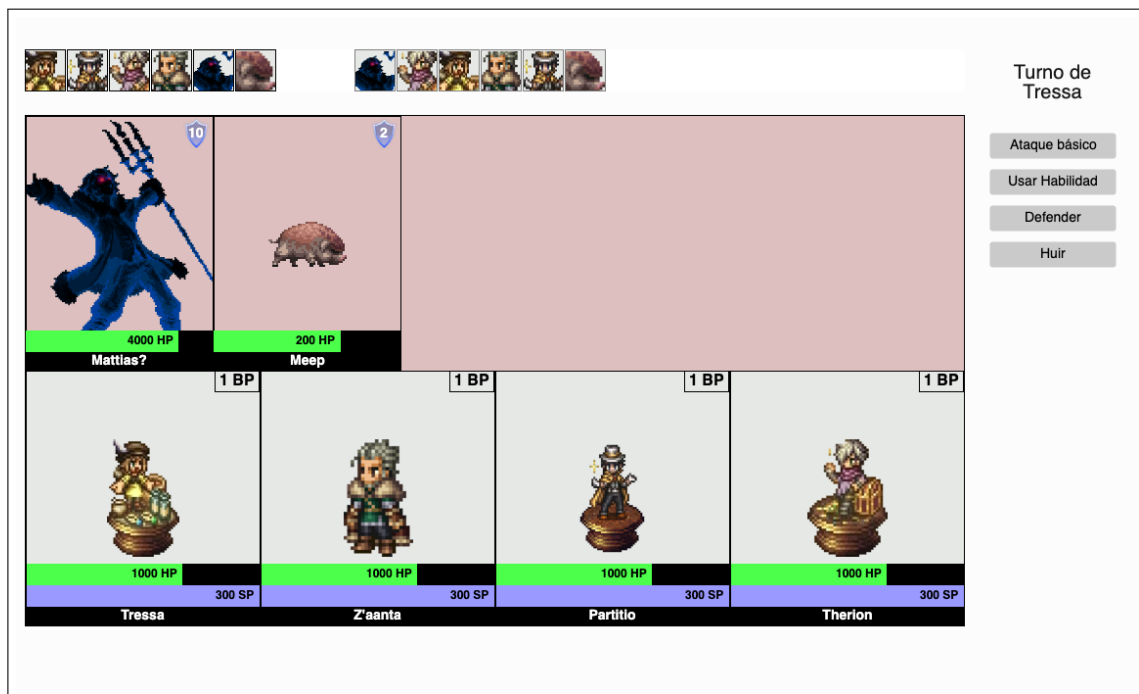
```
1 IClickedElement clickedElement;
2 do
3 {
4     clickedElement = gui.GetClickedElement();
5 } while (!(clickedElement.Type == ClickElementType.Button && clickedElement.Text == "Ataque
    básico"));
6
7 do {
8     clickedElement = gui.GetClickedElement();
9 } while (clickedElement.Type != ClickElementType.Beast);
10
```

```

11 if (clickedElement.Text == "Meep")
12 {
13     meep.HP -= 100;
14 }
15 else if (clickedElement.Text == "Mattias?")
16 {
17     mattias.HP -= 50;
18     mattias.Shields -= 1;
19 }
20 gui.Update(state);

```

Luego de dar *click* en “Ataque básico” y en alguna de las dos bestias en el tablero, el tablero se actualizará. Suponiendo que le dimos *click* a Meep, la vista se verá así:



Cabe destacar que si se da *click* en algún elemento distinto al botón “Ataque básico”, el programa se quedará en el primer bucle. Esto te servirá para garantizar que no pase nada si el usuario selecciona un elemento incorrecto.

Por último, para finalizar la partida se tiene el método `void ShowWinner(IWinner winner)` que mostrará una vista con el fin del juego dependiendo de la información que tenga `winner`. El equipo que se le entregue para mostrar depende de la circunstancia bajo la que se gane, por ejemplo, si gana el equipo de las bestias se le deberá pasar los nombres de las bestias en el equipo; en cambio, si los viajeros huyen, se le deberá pasar el equipo de los viajeros.

Supongamos que creamos la siguiente clase para utilizar la interfaz `IWinner`:

```

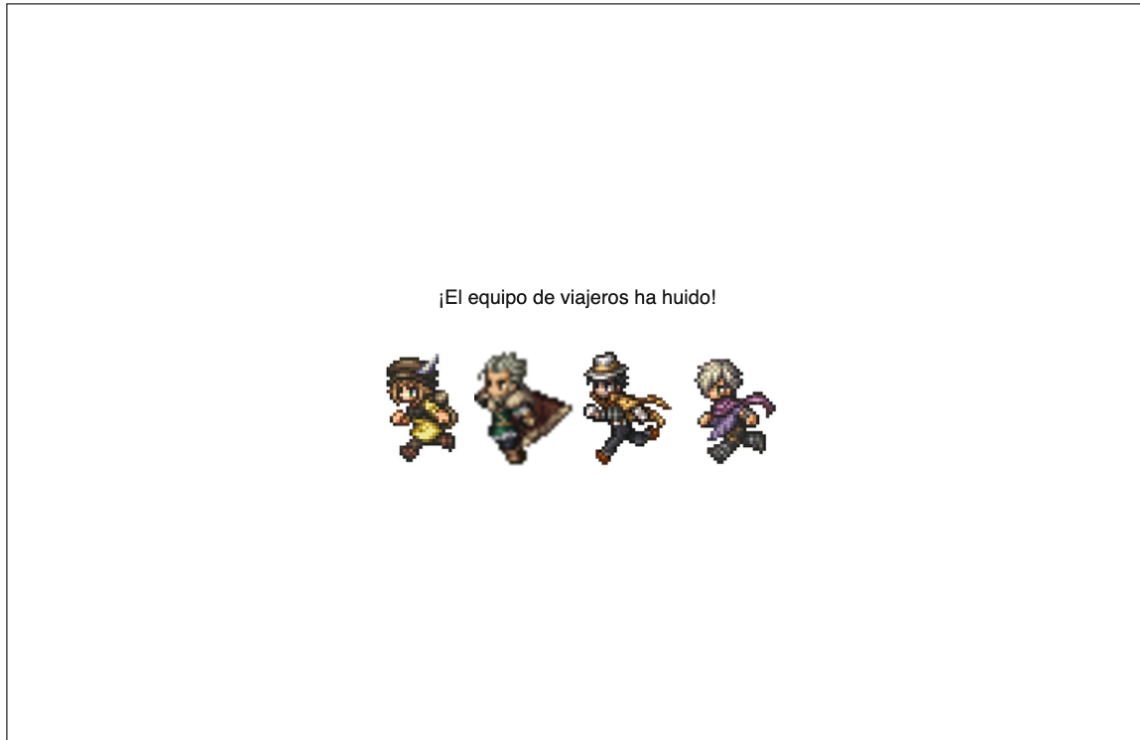
1 public class Winner(WinnerOption winnerOption, IEnumerable<string> team)
2     : IWinner
3 {
4     public WinnerOption WinnerOption { get; set; } = winnerOption;
5     public IEnumerable<string> Team { get; set; } = team;
6 }

```

Ahora, modificamos el método `Main()` eliminando lo anterior y agregando las siguientes líneas:

```
1 List<string> team = ["Tressa", "Z'aanta", "Partitio", "Therion"];
2 Winner winner = new Winner(WinnerOption.RunAway, team);
3 gui.ShowWinner(winner);
```

Este código mostrará esta ventana:



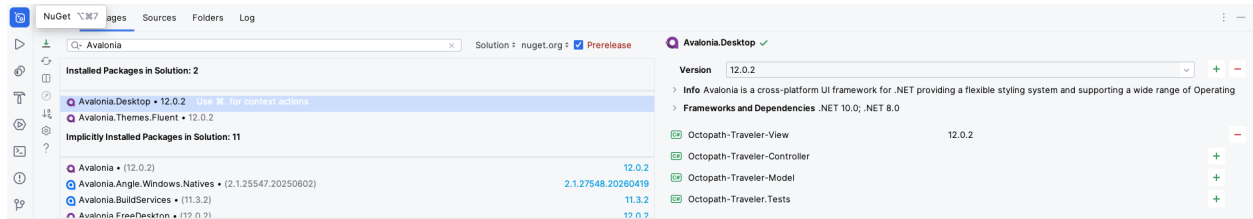
Consideraciones para la entrega

Cuando agregues la interfaz gráfica a tu entrega, debes cumplir con los siguientes requerimientos:

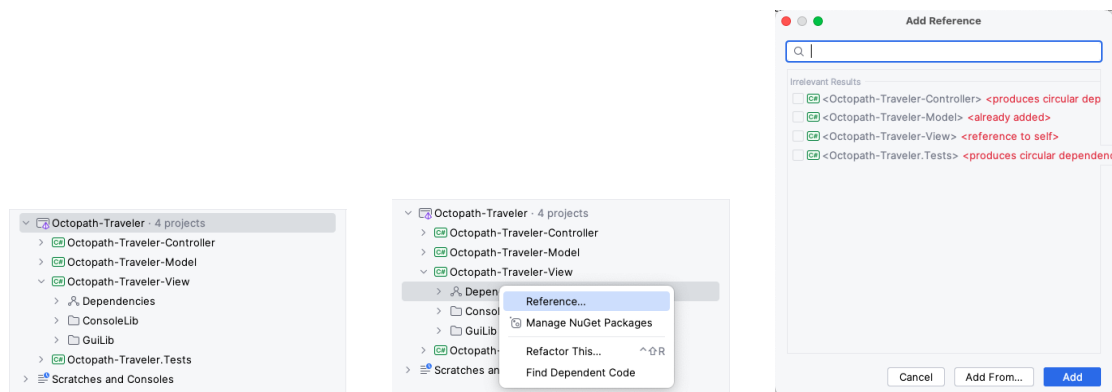
- Se puede elegir los equipos de los viajeros y las bestias.
- Si alguno de los equipos es inválido, se muestra el mensaje de que el equipo es inválido.
- Si ambos equipos son válidos, se muestra la ventana con ambos equipos junto sus stats, ambas colas de turnos y las opciones correspondientes a la unidad inicial.
- Se permite realizar todas las acciones disponibles para cada unidad, como atacar, usar habilidad, etc.
- Durante el juego se actualiza correctamente la ventana en base a las acciones del usuario, como la actualización de stats, las acciones disponibles, turnos, etc.
- Al finalizar el juego, se felicita al equipo que gana o se muestra a los viajeros que huyen.

A. ¿Cómo agregar la librería a mi proyecto?

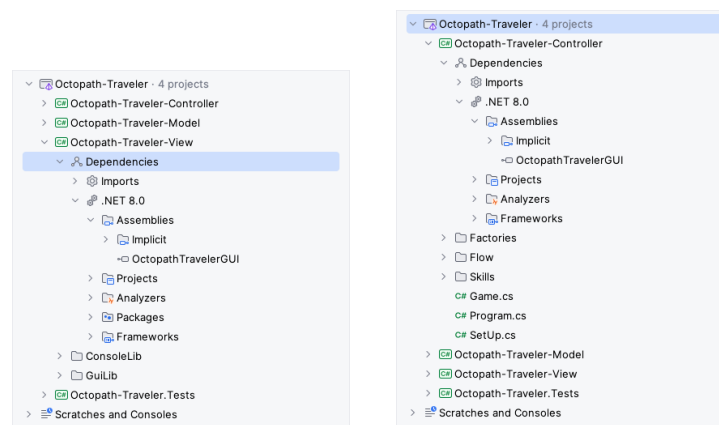
Para agregar la librería a un proyecto existente debes realizar los siguientes pasos. Primero, abre NuGet desde Rider e instala las librerías Avalonia, Avalonia.Desktop, Avalonia.Themes.Fluent y CommunityToolkit.Mvvm en el proyecto Octopath-Traveler-View:



Pon la carpeta GuiLib dentro del proyecto Octopath-Traveler-View. Luego en *Dependencies* selecciona que quieres agregar una referencia mediante “Add From...”



Aparecerá un menú con las carpetas de tu computador. Anda hasta la carpeta GuiLib dentro de tu proyecto Octopath-Traveler-View y selecciona OctopathTravelerGUI.dll. **Repite el proceso para el proyecto Octopath-Traveler-Controller.** Si seguiste los pasos correctamente, podrás ver que la librería OctopathTravelerGUI fue agregada a las dependencias de los proyectos Controller y View.



Finalmente, puedes verificar que la librería fue importada utilizando el siguiente código (en tu Program.cs):

```

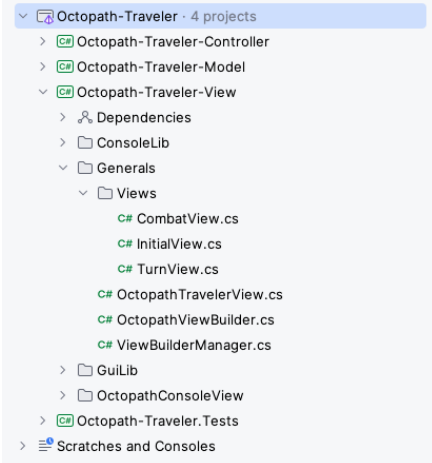
1 using Octopath_Traveler_GUI;
2
3 OTGUI gui = new OTGUI();
4 gui.Start(Main);
5
6 void Main() {}

```

Al correr ese código debería aparecer la siguiente ventana:

B. ¿Cómo incluyo la interfaz sin echarme los test cases?

Comencemos desde una entrega que implementa MVC. Como ejemplo, utilizemos una entrega que permite elegir el archivo de equipos y luego, arbitrariamente, dice que el equipo es inválido.



```

1 public class Game {
2     private readonly OctopathTravelerView _view;
3     private readonly string _teamsFolder;
4
5     public Game(View view, string teamsFolder) {
6         _view = new OctopathTravelerView(view);
7         _teamsFolder = teamsFolder;
8     }
9
10    public void Play() {
11        string teamFile = _view.SelectTeamFile(
12            _teamsFolder);
13        _view.AnnounceThatTeamIsInvalid();
14    }
15 }

```

El objetivo es agregar la interfaz gráfica sin echar a perder los test cases. Lo primero es notar que `Game` usa como vista `OctopathTravelerView`. Esta clase se encuentra en el proyecto de la vista, e incluye el siguiente código:

```

1 public class OctopathTravelerView(View view)
2 {
3     public void AnnounceThatTeamIsInvalid()
4         => view.WriteLine("Archivo de equipos inválido");
5
6     public string SelectTeamFile(string teamsFolder){
7         TeamSelectionView teamSelectionView = new (view, teamsFolder);
8         return teamSelectionView.SelectTeamFile();
9     }
10 }

```

Básicamente, tiene un método para anunciar que el equipo es inválido y otro para mostrar el menú que permite seleccionar al equipo. Notar que la clase `OctopathTravelerView` es nuestra vista consola. Contiene métodos que, mediante la consola, muestran mensajes y reciben inputs. Al mismo tiempo, esta clase permite que funcionen los test cases pues interactúa con el `View` utilizado para testear el proyecto.

Para poder cambiar al modo interfaz gráfica tenemos que crear una nueva vista. La nueva vista debería tener los mismos métodos que `OctopathTravelerView`, pero al elegir equipos y mostrar mensajes se debería hacer mediante la interfaz gráfica. De esa forma, el mismo controlador podrá funcionar con ambas vistas.

Partamos creando una interfaz con los métodos comunes a todas las vistas que el controlador podrá utilizar.

```

1 public interface IOctopathTravelerView{
2     void AnnounceThatTeamIsInvalid();
3     string SelectTeamFile(string teamsFolder);
4 }

```

Ahora hacemos que nuestra vista actual implemente la interfaz:

```

1 public class OctopathTravelerView(View view) : IOctopathTravelerView {
2     public void AnnounceThatTeamIsInvalid()
3         => view.WriteLine("Archivo de equipos inválido");
4
5     public string SelectTeamFile(string teamsFolder){
6         TeamSelectionView teamSelectionView = new (view, teamsFolder);
7         return teamSelectionView.SelectTeamFile();
8     }
9 }

```

Finalmente, hacemos que Game ahora funcione con cualquier vista que implemente nuestra interfaz:

```
1 public class Game {
2     private readonly IOctopathTravelerView _view;
3     private readonly string _teamsFolder;
4
5     public Game(View view, string teamsFolder) {
6         _view = new OctopathTravelerView(view);
7         _teamsFolder = teamsFolder;
8     }
9
10    public void Play() {
11        string teamFile = _view.SelectTeamFile(_teamsFolder);
12        _view.AnnounceThatTeamIsInvalid();
13    }
14 }
```

En el proyecto Octopath-Traveler-View, agregamos una nueva vista para la interfaz gráfica:

```
1 public class OctopathTravelerGUIView : IOctopathTravelerView {
2     private OTGUI _window = new ();
3
4     public void Start(Action startProgram)
5         => _window.Start(startProgram);
6
7     public void AnnounceThatTeamIsInvalid()
8         => _window.ShowInvalidTeams();
9
10    public string SelectTeamFile(string teamsFolder){
11        List<string> travelers = _window.GetTravelersTeam();
12        List<string> beasts = _window.GetBeastsTeam();
13        return TeamInfoFormatter.FormatTeamInfo(travelers, beasts);
14    }
15 }
```

Gracias a que esta vista también implementa OctopathTravelerView, Game podrá usarla sin problemas. Aunque, para que ello ocurra, tenemos que crear un objeto OctopathTravelerGUIView. La forma más simple de hacer esto, sin tener que cambiar los test cases, es creando un segundo constructor para Game.

```
1 public class Game {
2     private readonly IOctopathTravelerView _view;
3     private readonly string _teamsFolder;
4
5     public Game(View view, string teamsFolder) {
6         _view = new OctopathTravelerView(view);
7         _teamsFolder = teamsFolder;
8     }
9
10    public Game(OctopathTravelerGUIView view){
11        _view = view;
12        _teamsFolder = "";
13    }
14
15    public void Play() {
16        string teamFile = _view.SelectTeamFile(_teamsFolder);
17        _view.AnnounceThatTeamIsInvalid();
18    }
19 }
```

Finalmente, en el Program.cs podemos decidir si comenzamos en modo consola o en modo interfaz gráfica dependiendo del constructor que usamos.

```
1 using Octopath_Traveler_GUI;
```

```

2 using Octopath_Traveler_View;
3 using Octopath_Traveler;
4
5 bool useGui = true;
6 if (useGui)
7 {
8     OctopathTravelerGUIView view = new ();
9     Game game = new Game(view);
10    view.Start(game.Play);
11 }
12 else
13 {
14     string testFolder = SelectTestFolder();
15     string test = SelectTest(testFolder);
16     string teamsFolder = testFolder.Replace("-Tests", "");
17     AnnounceTestCase(test);
18
19     var view = View.BuildManualTestingView(test);
20     var game = new Game(view, teamsFolder);
21     game.Play();
22 }

```